

Wish list for functional analysis package

Jon Brodziak and Matthew Supernaw

2025-19-12

Wish list for functional analysis package

Purpose

This wish list focuses on improving how the functional analysis code samples a simplex input space, evaluates an objective function and its derivatives, and summarizes smoothness, stability, and parameter interdependence in a way that is interpretable and numerically correct.

Should have

1) Simplex input support

- Read and process a simplex file with 3 proportions (a 2-D simplex with a sum-to-1 constraint).
- Set `dirichlet_default()` to work with the **vector** of Dirichlet multinomial concentration parameters.
- Run for 3 Dirichlet multinomial forms: default (unconstrained alpha concentration vector), saturated (proportion vector and total concentration scalar), and linear (proportion vector and linear sample size scaling coefficient)

2) Core output quantities at each sampled point

For each simplex point $\mathbf{x}$, output a row containing:

- **Objective value**: the log-likelihood / objective function value $f(\mathbf{x})$.
- **Gradient**: $\nabla f(\mathbf{x})$, i.e., first derivatives w.r.t. each input parameter used by the analysis.
- **Continuity / smoothness diagnostics** (see “Stochasticity and smoothness” below).
- **Local curvature summaries** (when requested): **Hessian** $H(\mathbf{x})$ and/or its inverse $H(\mathbf{x})^{-1}$ (see covariance note below).
- **Bounds**: the minimum and maximum objective values observed over the sampled input space, along with the argmin/argmax input points.

3) Output file formats

- A raw output file with **rows = simplex points**, including:
 - input parameters
 - objective value
 - gradients
 - optional Hessian / inverse Hessian summaries
 - A summary output file with:
 - min/max and locations
 - smoothness/stochasticity summaries
 - covariance/correlation summaries at selected points (or at a user-specified point)
-

Correctness and interpretation fixes (small changes)

A) Fix the definition of differences used for “stochasticity”

- Current behavior uses a fixed baseline value inside `Diff()` (not a standard lagged difference).
- **Change requested:** define a conventional lagged difference:
 - $\Delta f_k = f(\mathbf{x}^{(k)}) - f(\mathbf{x}^{(k-1)})$ for lag 1 (or $f_k - f_{k-\ell}$ for lag ℓ).
- This makes “stochasticity” based on changes between adjacent samples *in the chosen ordering*.

B) Fix the correlation denominator

- Current “correlation” normalization can yield values outside $[-1, 1]$ even when the underlying matrix is valid.
- **Change requested:** compute standard correlation:
 - $\rho_{ij} = \text{cov}_{ij} / \sqrt{\text{cov}_{ii} \text{cov}_{jj}}$.
- Add guards for zero/invalid diagonal elements so undefined correlations become NA/NaN.

C) Add a stability guard for near-zero mean differences

- If $\text{Mean}(\Delta f) \approx 0$, then $\text{SD}(\Delta f) / |\text{Mean}(\Delta f)|$ can blow up.
- **Change requested:** return NA/NaN (or use a more stable denominator such as $\text{Mean}(|\Delta f|)$) when the mean is too close to zero.

Stochasticity and smoothness (make it reflect input space)

Option B (preferred): input-space nearest-neighbor smoothness

The goal is to measure smoothness with respect to **distance in input space**, not the arbitrary enumeration order of simplex points.

- Define a (scaled) input-space distance:
 - $d(\mathbf{x}, \mathbf{x}') = \sqrt{\sum_i ((x_i - x'_i)/s_i)^2}$, where s_i is a scale (e.g., parameter range).
- For each point $\mathbf{x}^{(k)}$, find its nearest neighbor $\mathbf{x}^{(\text{NN}(k))}$.
- Compute local “slope” magnitudes:
 - $s_k = (f(\mathbf{x}^{(k)}) - f(\mathbf{x}^{(\text{NN}(k))})) / d(\mathbf{x}^{(k)}, \mathbf{x}^{(\text{NN}(k))})$.
- Summarize smoothness using a CV-like statistic:
 - **Input-space stochasticity:** $\text{SD}(s) / |\text{Mean}(s)|$ (or $\text{SD}(s) / \text{Mean}(|s|)$ for more stability).
- Apply the same nearest-neighbor slope idea to each gradient component to diagnose noisy derivatives:
 - $s_{k,i} = (\partial f / \partial x_i|_k - \partial f / \partial x_i|_{\text{NN}(k)}) / d(\cdot)$.

Implementation notes: - A simple 1-NN search is $O(K^2)$ and is acceptable for small-to-moderate K . - For large K , consider a kd-tree / ANN library to reduce runtime. - Provide a user option such as: + `stochasticity_mode = c("sequence", "nearest_neighbor")` + and keep “sequence” mode available for backwards compatibility.

Interpretation: - Low input-space stochasticity indicates a smooth, deterministic mapping over nearby points. - High input-space stochasticity suggests discontinuities, numerical instability, or genuine stochastic behavior not explained by inputs.

Covariance, correlation, and bounds (interpretation expectations)

Covariance / curvature

- If “covariance” is computed as the inverse Hessian $H^{-1}(\mathbf{x})$, then it should be described as a **local curvature-based approximation**.

- Interpretation is most meaningful near an optimum (e.g., for likelihood-based inference). Otherwise, treat it as “local curvature inverse,” not an inferential covariance.

Bounds

- Clarify whether “bounds” mean:
 - min/max over the sampled design, or
 - analytic bounds implied by constraints.
 - At minimum, report min/max over sampled points plus the argmin/argmax input vectors.
-

Would like to have (next tier)

General simplex dimension

- Extend to simplex files with N proportions (general simplex).

Additional outputs

- Divergence of the function gradient at a point (or other curvature diagnostics).
- The ability to create **replicates at a single input point** to quantify true stochastic output variance.
- The ability to choose which output variables are stored (objective only vs objective + derivatives + curvature).
- A log file capturing run settings, timing, and warnings (e.g., non-invertible Hessians, NaNs, ill-conditioned curvature).

Usability

- Clear documentation of how points are sampled, how derivatives are computed (AD mode), and how each summary statistic is defined.
- Unit tests for:
 - `Diff()` correctness
 - correlation scaling producing values in $[-1, 1]$ when defined
 - nearest-neighbor stochasticity on known smooth functions